

Stream



Event-driven readable and writable streams for non-blocking I/O in ReactPHP.

In order to make the EventLoop easier to use, this component introduces the powerful concept of “streams”. Streams allow you to efficiently process huge amounts of data (such as a multi Gigabyte file download) in small chunks without having to store everything in memory at once. They are very similar to the streams found in PHP itself, but have an interface more suited for async, non-blocking I/O.

Table of contents

- Stream usage
 - ReadableStreamInterface
 - * data event
 - * end event
 - * error event
 - * close event
 - * isReadable()
 - * pause()
 - * resume()
 - * pipe()
 - * close()
 - WritableStreamInterface
 - * drain event
 - * pipe event
 - * error event
 - * close event
 - * isWritable()
 - * write()
 - * end()
 - * close()
 - DuplexStreamInterface
- Creating streams
 - ReadableResourceStream
 - WritableResourceStream
 - DuplexResourceStream
 - ThroughStream
 - CompositeStream
- Usage
- Install
- Tests
- License
- More

Stream usage

ReactPHP uses the concept of “streams” throughout its ecosystem to provide a consistent higher-level abstraction for processing streams of arbitrary data contents and size. While a stream itself is a quite low-level concept, it can be used as a powerful abstraction to build higher-level components and protocols on top.

If you’re new to this concept, it helps to think of them as a water pipe: You can consume water from a source or you can produce water and forward (pipe) it to any destination (sink).

Similarly, streams can either be

- readable (such as `STDIN` terminal input) or
- writable (such as `STDOUT` terminal output) or
- duplex (both readable *and* writable, such as a TCP/IP connection)

Accordingly, this package defines the following three interfaces

- `ReadableStreamInterface`
- `WritableStreamInterface`
- `DuplexStreamInterface`

ReadableStreamInterface

The `ReadableStreamInterface` is responsible for providing an interface for read-only streams and the readable side of duplex streams.

Besides defining a few methods, this interface also implements the `EventEmitterInterface` which allows you to react to certain events.

The event callback functions **MUST** be a valid `callable` that obeys strict parameter definitions and **MUST** accept event parameters exactly as documented. The event callback functions **MUST NOT** throw an `Exception`. The return value of the event callback functions will be ignored and has no effect, so for performance reasons you’re recommended to not return any excessive data structures.

Every implementation of this interface **MUST** follow these event semantics in order to be considered a well-behaving stream.

Note that higher-level implementations of this interface may choose to define additional events with dedicated semantics not defined as part of this low-level stream specification. Conformance with these event semantics is out of scope for this interface, so you may also have to refer to the documentation of such a higher-level implementation.

data event The `data` event will be emitted whenever some data was read/received from this source stream. The event receives a single mixed argument for incoming data.

```
$stream->on('data', function ($data) {
    echo $data;
});
```

This event MAY be emitted any number of times, which may be zero times if this stream does not send any data at all. It SHOULD not be emitted after an **end** or **close** event.

The given **\$data** argument may be of mixed type, but it's usually recommended it SHOULD be a **string** value or MAY use a type that allows representation as a **string** for maximum compatibility.

Many common streams (such as a TCP/IP connection or a file-based stream) will emit the raw (binary) payload data that is received over the wire as chunks of **string** values.

Due to the stream-based nature of this, the sender may send any number of chunks with varying sizes. There are no guarantees that these chunks will be received with the exact same framing the sender intended to send. In other words, many lower-level protocols (such as TCP/IP) transfer the data in chunks that may be anywhere between single-byte values to several dozens of kilobytes. You may want to apply a higher-level protocol to these low-level data chunks in order to achieve proper message framing.

end event The **end** event will be emitted once the source stream has successfully reached the end of the stream (EOF).

```
$stream->on('end', function () {
    echo 'END';
});
```

This event SHOULD be emitted once or never at all, depending on whether a successful end was detected. It SHOULD NOT be emitted after a previous **end** or **close** event. It MUST NOT be emitted if the stream closes due to a non-successful end, such as after a previous **error** event.

After the stream is ended, it MUST switch to non-readable mode, see also **isReadable()**.

This event will only be emitted if the *end* was reached successfully, not if the stream was interrupted by an unrecoverable error or explicitly closed. Not all streams know this concept of a “successful end”. Many use-cases involve detecting when the stream closes (terminates) instead, in this case you should use the **close** event. After the stream emits an **end** event, it SHOULD usually be followed by a **close** event.

Many common streams (such as a TCP/IP connection or a file-based stream) will emit this event if either the remote side closes the connection or a file handle was successfully read until reaching its end (EOF).

Note that this event should not be confused with the `end()` method. This event defines a successful end *reading* from a source stream, while the `end()` method defines *writing* a successful end to a destination stream.

error event The **error** event will be emitted once a fatal error occurs, usually while trying to read from this stream. The event receives a single `Exception` argument for the error instance.

```
$server->on('error', function (Exception $e) {  
    echo 'Error: ' . $e->getMessage() . PHP_EOL;  
});
```

This event SHOULD be emitted once the stream detects a fatal error, such as a fatal transmission error or after an unexpected **data** or premature **end** event. It SHOULD NOT be emitted after a previous **error**, **end** or **close** event. It MUST NOT be emitted if this is not a fatal error condition, such as a temporary network issue that did not cause any data to be lost.

After the stream errors, it MUST close the stream and SHOULD thus be followed by a **close** event and then switch to non-readable mode, see also `close()` and `isReadable()`.

Many common streams (such as a TCP/IP connection or a file-based stream) only deal with data transmission and do not make assumption about data boundaries (such as unexpected **data** or premature **end** events). In other words, many lower-level protocols (such as TCP/IP) may choose to only emit this for a fatal transmission error once and will then close (terminate) the stream in response.

If this stream is a `DuplexStreamInterface`, you should also notice how the writable side of the stream also implements an **error** event. In other words, an error may occur while either reading or writing the stream which should result in the same error processing.

close event The **close** event will be emitted once the stream closes (terminates).

```
$stream->on('close', function () {  
    echo 'CLOSED';  
});
```

This event SHOULD be emitted once or never at all, depending on whether the stream ever terminates. It SHOULD NOT be emitted after a previous **close** event.

After the stream is closed, it MUST switch to non-readable mode, see also `isReadable()`.

Unlike the **end** event, this event SHOULD be emitted whenever the stream closes, irrespective of whether this happens implicitly due to an unrecoverable

error or explicitly when either side closes the stream. If you only want to detect a *successful* end, you should use the **end** event instead.

Many common streams (such as a TCP/IP connection or a file-based stream) will likely choose to emit this event after reading a *successful* **end** event or after a fatal transmission **error** event.

If this stream is a **DuplexStreamInterface**, you should also notice how the writable side of the stream also implements a **close** event. In other words, after receiving this event, the stream MUST switch into non-writable AND non-readable mode, see also **isWritable()**. Note that this event should not be confused with the **end** event.

isReadable() The **isReadable(): bool** method can be used to check whether this stream is in a readable state (not closed already).

This method can be used to check if the stream still accepts incoming data events or if it is ended or closed already. Once the stream is non-readable, no further **data** or **end** events SHOULD be emitted.

```
assert($stream->isReadable() === false);
```

```
$stream->on('data', assertNeverCalled());  
$stream->on('end', assertNeverCalled());
```

A successfully opened stream always MUST start in readable mode.

Once the stream ends or closes, it MUST switch to non-readable mode. This can happen any time, explicitly through **close()** or implicitly due to a remote close or an unrecoverable transmission error. Once a stream has switched to non-readable mode, it MUST NOT transition back to readable mode.

If this stream is a **DuplexStreamInterface**, you should also notice how the writable side of the stream also implements an **isWritable()** method. Unless this is a half-open duplex stream, they SHOULD usually have the same return value.

pause() The **pause(): void** method can be used to pause reading incoming data events.

Removes the data source file descriptor from the event loop. This allows you to throttle incoming data.

Unless otherwise noted, a successfully opened stream SHOULD NOT start in paused state.

Once the stream is paused, no further **data** or **end** events SHOULD be emitted.

```
$stream->pause();
```

```
$stream->on('data', assertShouldNeverCalled());
$stream->on('end', assertShouldNeverCalled());
```

This method is advisory-only, though generally not recommended, the stream MAY continue emitting **data** events.

You can continue processing events by calling **resume()** again.

Note that both methods can be called any number of times, in particular calling **pause()** more than once SHOULD NOT have any effect.

See also **resume()**.

resume() The **resume(): void** method can be used to resume reading incoming data events.

Re-attach the data source after a previous **pause()**.

```
$stream->pause();
```

```
Loop::addTimer(1.0, function () use ($stream) {
    $stream->resume();
});
```

Note that both methods can be called any number of times, in particular calling **resume()** without a prior **pause()** SHOULD NOT have any effect.

See also **pause()**.

pipe() The **pipe(WritableStreamInterface \$dest, array \$options = [])** method can be used to pipe all the data from this readable source into the given writable destination.

Automatically sends all incoming data to the destination. Automatically throttles the source based on what the destination can handle.

```
$source->pipe($dest);
```

Similarly, you can also pipe an instance implementing **DuplexStreamInterface** into itself in order to write back all the data that is received. This may be a useful feature for a TCP/IP echo service:

```
$connection->pipe($connection);
```

This method returns the destination stream as-is, which can be used to set up chains of piped streams:

```
$source->pipe($decodeGzip)->pipe($filterBadWords)->pipe($dest);
```

By default, this will call **end()** on the destination stream once the source stream emits an **end** event. This can be disabled like this:

```
$source->pipe($dest, array('end' => false));
```

Note that this only applies to the **end** event. If an **error** or explicit **close** event happens on the source stream, you'll have to manually close the destination stream:

```
$source->pipe($dest);
$source->on('close', function () use ($dest) {
    $dest->end('BYE!');
});
```

If the source stream is not readable (closed state), then this is a NO-OP.

```
$source->close();
$source->pipe($dest); // NO-OP
```

If the destination stream is not writable (closed state), then this will simply throttle (pause) the source stream:

```
$dest->close();
$source->pipe($dest); // calls $source->pause()
```

Similarly, if the destination stream is closed while the pipe is still active, it will also throttle (pause) the source stream:

```
$source->pipe($dest);
$dest->close(); // calls $source->pause()
```

Once the pipe is set up successfully, the destination stream **MUST** emit a **pipe** event with this source stream as an event argument.

close() The **close(): void** method can be used to close the stream (forcefully).

This method can be used to (forcefully) close the stream.

```
$stream->close();
```

Once the stream is closed, it **SHOULD** emit a **close** event. Note that this event **SHOULD NOT** be emitted more than once, in particular if this method is called multiple times.

After calling this method, the stream **MUST** switch into a non-readable mode, see also **isReadable()**. This means that no further **data** or **end** events **SHOULD** be emitted.

```
$stream->close();
assert($stream->isReadable() === false);

$stream->on('data', assertNeverCalled());
$stream->on('end', assertNeverCalled());
```

If this stream is a **DuplexStreamInterface**, you should also notice how the writable side of the stream also implements a **close()** method. In other words,

after calling this method, the stream MUST switch into non-writable AND non-readable mode, see also `isWritable()`. Note that this method should not be confused with the `end()` method.

WritableStreamInterface

The `WritableStreamInterface` is responsible for providing an interface for write-only streams and the writable side of duplex streams.

Besides defining a few methods, this interface also implements the `EventEmitterInterface` which allows you to react to certain events.

The event callback functions MUST be a valid `callable` that obeys strict parameter definitions and MUST accept event parameters exactly as documented. The event callback functions MUST NOT throw an `Exception`. The return value of the event callback functions will be ignored and has no effect, so for performance reasons you're recommended to not return any excessive data structures.

Every implementation of this interface MUST follow these event semantics in order to be considered a well-behaving stream.

Note that higher-level implementations of this interface may choose to define additional events with dedicated semantics not defined as part of this low-level stream specification. Conformance with these event semantics is out of scope for this interface, so you may also have to refer to the documentation of such a higher-level implementation.

drain event The `drain` event will be emitted whenever the write buffer became full previously and is now ready to accept more data.

```
$stream->on('drain', function () use ($stream) {  
    echo 'Stream is now ready to accept more data';  
});
```

This event SHOULD be emitted once every time the buffer became full previously and is now ready to accept more data. In other words, this event MAY be emitted any number of times, which may be zero times if the buffer never became full in the first place. This event SHOULD NOT be emitted if the buffer has not become full previously.

This event is mostly used internally, see also `write()` for more details.

pipe event The `pipe` event will be emitted whenever a readable stream is `pipe()`d into this stream. The event receives a single `ReadableStreamInterface` argument for the source stream.

```
$stream->on('pipe', function (ReadableStreamInterface $source) use ($stream) {  
    echo 'Now receiving piped data';  
});
```



```

    // explicitly close target if source emits an error
    $source->on('error', function () use ($stream) {
        $stream->close();
    });
});

```

```

$source->pipe($stream);

```

This event **MUST** be emitted once for each readable stream that is successfully piped into this destination stream. In other words, this event **MAY** be emitted any number of times, which may be zero times if no stream is ever piped into this stream. This event **MUST NOT** be emitted if either the source is not readable (closed already) or this destination is not writable (closed already).

This event is mostly used internally, see also `pipe()` for more details.

error event The **error** event will be emitted once a fatal error occurs, usually while trying to write to this stream. The event receives a single **Exception** argument for the error instance.

```

$stream->on('error', function (Exception $e) {
    echo 'Error: ' . $e->getMessage() . PHP_EOL;
});

```

This event **SHOULD** be emitted once the stream detects a fatal error, such as a fatal transmission error. It **SHOULD NOT** be emitted after a previous **error** or **close** event. It **MUST NOT** be emitted if this is not a fatal error condition, such as a temporary network issue that did not cause any data to be lost.

After the stream errors, it **MUST** close the stream and **SHOULD** thus be followed by a **close** event and then switch to non-writable mode, see also `close()` and `isWritable()`.

Many common streams (such as a TCP/IP connection or a file-based stream) only deal with data transmission and may choose to only emit this for a fatal transmission error once and will then close (terminate) the stream in response.

If this stream is a **DuplexStreamInterface**, you should also notice how the readable side of the stream also implements an **error** event. In other words, an error may occur while either reading or writing the stream which should result in the same error processing.

close event The **close** event will be emitted once the stream closes (terminates).

```

$stream->on('close', function () {
    echo 'CLOSED';
});

```

This event SHOULD be emitted once or never at all, depending on whether the stream ever terminates. It SHOULD NOT be emitted after a previous `close` event.

After the stream is closed, it MUST switch to non-writable mode, see also `isWritable()`.

This event SHOULD be emitted whenever the stream closes, irrespective of whether this happens implicitly due to an unrecoverable error or explicitly when either side closes the stream.

Many common streams (such as a TCP/IP connection or a file-based stream) will likely choose to emit this event after flushing the buffer from the `end()` method, after receiving a *successful* `end` event or after a fatal transmission `error` event.

If this stream is a `DuplexStreamInterface`, you should also notice how the readable side of the stream also implements a `close` event. In other words, after receiving this event, the stream MUST switch into non-writable AND non-readable mode, see also `isReadable()`. Note that this event should not be confused with the `end` event.

isWritable() The `isWritable(): bool` method can be used to check whether this stream is in a writable state (not closed already).

This method can be used to check if the stream still accepts writing any data or if it is ended or closed already. Writing any data to a non-writable stream is a NO-OP:

```
assert($stream->isWritable() === false);
```

```
$stream->write('end'); // NO-OP
$stream->end('end'); // NO-OP
```

A successfully opened stream always MUST start in writable mode.

Once the stream ends or closes, it MUST switch to non-writable mode. This can happen any time, explicitly through `end()` or `close()` or implicitly due to a remote close or an unrecoverable transmission error. Once a stream has switched to non-writable mode, it MUST NOT transition back to writable mode.

If this stream is a `DuplexStreamInterface`, you should also notice how the readable side of the stream also implements an `isReadable()` method. Unless this is a half-open duplex stream, they SHOULD usually have the same return value.

write() The `write(mixed $data): bool` method can be used to write some data into the stream.

A successful write **MUST** be confirmed with a boolean **true**, which means that either the data was written (flushed) immediately or is buffered and scheduled for a future write. Note that this interface gives you no control over explicitly flushing the buffered data, as finding the appropriate time for this is beyond the scope of this interface and left up to the implementation of this interface.

Many common streams (such as a TCP/IP connection or file-based stream) may choose to buffer all given data and schedule a future flush by using an underlying EventLoop to check when the resource is actually writable.

If a stream cannot handle writing (or flushing) the data, it **SHOULD** emit an **error** event and **MAY** **close()** the stream if it can not recover from this error.

If the internal buffer is full after adding **\$data**, then **write()** **SHOULD** return **false**, indicating that the caller should stop sending data until the buffer drains. The stream **SHOULD** send a **drain** event once the buffer is ready to accept more data.

Similarly, if the stream is not writable (already in a closed state) it **MUST NOT** process the given **\$data** and **SHOULD** return **false**, indicating that the caller should stop sending data.

The given **\$data** argument **MAY** be of mixed type, but it's usually recommended it **SHOULD** be a **string** value or **MAY** use a type that allows representation as a **string** for maximum compatibility.

Many common streams (such as a TCP/IP connection or a file-based stream) will only accept the raw (binary) payload data that is transferred over the wire as chunks of **string** values.

Due to the stream-based nature of this, the sender may send any number of chunks with varying sizes. There are no guarantees that these chunks will be received with the exact same framing the sender intended to send. In other words, many lower-level protocols (such as TCP/IP) transfer the data in chunks that may be anywhere between single-byte values to several dozens of kilobytes. You may want to apply a higher-level protocol to these low-level data chunks in order to achieve proper message framing.

end() The **end(mixed \$data = null): void** method can be used to successfully end the stream (after optionally sending some final data).

This method can be used to successfully end the stream, i.e. close the stream after sending out all data that is currently buffered.

```
$stream->write('hello');  
$stream->write('world');  
$stream->end();
```

If there's no data currently buffered and nothing to be flushed, then this method **MAY** **close()** the stream immediately.

If there's still data in the buffer that needs to be flushed first, then this method SHOULD try to write out this data and only then `close()` the stream. Once the stream is closed, it SHOULD emit a `close` event.

Note that this interface gives you no control over explicitly flushing the buffered data, as finding the appropriate time for this is beyond the scope of this interface and left up to the implementation of this interface.

Many common streams (such as a TCP/IP connection or file-based stream) may choose to buffer all given data and schedule a future flush by using an underlying `EventLoop` to check when the resource is actually writable.

You can optionally pass some final data that is written to the stream before ending the stream. If a non-null value is given as `$data`, then this method will behave just like calling `write($data)` before ending with no data.

```
// shorter version
$stream->end('bye');

// same as longer version
$stream->write('bye');
$stream->end();
```

After calling this method, the stream MUST switch into a non-writable mode, see also `isWritable()`. This means that no further writes are possible, so any additional `write()` or `end()` calls have no effect.

```
$stream->end();
assert($stream->isWritable() === false);

$stream->write('nope'); // NO-OP
$stream->end(); // NO-OP
```

If this stream is a `DuplexStreamInterface`, calling this method SHOULD also end its readable side, unless the stream supports half-open mode. In other words, after calling this method, these streams SHOULD switch into non-writable AND non-readable mode, see also `isReadable()`. This implies that in this case, the stream SHOULD NOT emit any `data` or `end` events anymore. Streams MAY choose to use the `pause()` method logic for this, but special care may have to be taken to ensure a following call to the `resume()` method SHOULD NOT continue emitting readable events.

Note that this method should not be confused with the `close()` method.

`close()` The `close(): void` method can be used to close the stream (forcefully).

This method can be used to forcefully close the stream, i.e. close the stream without waiting for any buffered data to be flushed. If there's still data in the buffer, this data SHOULD be discarded.

```
$stream->close();
```

Once the stream is closed, it SHOULD emit a `close` event. Note that this event SHOULD NOT be emitted more than once, in particular if this method is called multiple times.

After calling this method, the stream MUST switch into a non-writable mode, see also `isWritable()`. This means that no further writes are possible, so any additional `write()` or `end()` calls have no effect.

```
$stream->close();
assert($stream->isWritable() === false);
```

```
$stream->write('nope'); // NO-OP
$stream->end(); // NO-OP
```

Note that this method should not be confused with the `end()` method. Unlike the `end()` method, this method does not take care of any existing buffers and simply discards any buffer contents. Likewise, this method may also be called after calling `end()` on a stream in order to stop waiting for the stream to flush its final data.

```
$stream->end();
Loop::addTimer(1.0, function () use ($stream) {
    $stream->close();
});
```

If this stream is a `DuplexStreamInterface`, you should also notice how the readable side of the stream also implements a `close()` method. In other words, after calling this method, the stream MUST switch into non-writable AND non-readable mode, see also `isReadable()`.

DuplexStreamInterface

The `DuplexStreamInterface` is responsible for providing an interface for duplex streams (both readable and writable).

It builds on top of the existing interfaces for readable and writable streams and follows the exact same method and event semantics. If you're new to this concept, you should look into the `ReadableStreamInterface` and `WritableStreamInterface` first.

Besides defining a few methods, this interface also implements the `EventEmitterInterface` which allows you to react to the same events defined on the `ReadableStreamInterface` and `WritableStreamInterface`.

The event callback functions MUST be a valid `callable` that obeys strict parameter definitions and MUST accept event parameters exactly as documented. The event callback functions MUST NOT throw an `Exception`. The return value of the event callback functions will be ignored and has no effect, so for

performance reasons you're recommended to not return any excessive data structures.

Every implementation of this interface **MUST** follow these event semantics in order to be considered a well-behaving stream.

Note that higher-level implementations of this interface may choose to define additional events with dedicated semantics not defined as part of this low-level stream specification. Conformance with these event semantics is out of scope for this interface, so you may also have to refer to the documentation of such a higher-level implementation.

See also `ReadableStreamInterface` and `WritableStreamInterface` for more details.

Creating streams

ReactPHP uses the concept of “streams” throughout its ecosystem, so that many higher-level consumers of this package only deal with stream usage. This implies that stream instances are most often created within some higher-level components and many consumers never actually have to deal with creating a stream instance.

- Use `react/socket` if you want to accept incoming or establish outgoing plaintext TCP/IP or secure TLS socket connection streams.
- Use `react/http` if you want to receive an incoming HTTP request body streams.
- Use `react/child-process` if you want to communicate with child processes via process pipes such as `STDIN`, `STDOUT`, `STDERR` etc.
- Use experimental `react/filesystem` if you want to read from / write to the filesystem.
- See also the last chapter for more real-world applications.

However, if you are writing a lower-level component or want to create a stream instance from a stream resource, then the following chapter is for you.

Note that the following examples use `fopen()` and `stream_socket_client()` for illustration purposes only. These functions **SHOULD NOT** be used in a truly async program because each call may take several seconds to complete and would block the EventLoop otherwise. Additionally, the `fopen()` call will return a file handle on some platforms which may or may not be supported by all EventLoop implementations. As an alternative, you may want to use higher-level libraries listed above.

ReadableResourceStream

The `ReadableResourceStream` is a concrete implementation of the `ReadableStreamInterface` for PHP's stream resources.

This can be used to represent a read-only resource like a file stream opened in readable mode or a stream such as STDIN:

```
$stream = new ReadableResourceStream(STDIN);
$stream->on('data', function ($chunk) {
    echo $chunk;
});
$stream->on('end', function () {
    echo 'END';
});
```

See also `ReadableStreamInterface` for more details.

The first parameter given to the constructor **MUST** be a valid stream resource that is opened in reading mode (e.g. `fopen()` mode `r`). Otherwise, it will throw an `InvalidArgumentException`:

```
// throws InvalidArgumentException
$stream = new ReadableResourceStream(false);
```

See also the `DuplexResourceStream` for read-and-write stream resources otherwise.

Internally, this class tries to enable non-blocking mode on the stream resource which may not be supported for all stream resources. Most notably, this is not supported by pipes on Windows (STDIN etc.). If this fails, it will throw a `RuntimeException`:

```
// throws RuntimeException on Windows
$stream = new ReadableResourceStream(STDIN);
```

Once the constructor is called with a valid stream resource, this class will take care of the underlying stream resource. You **SHOULD** only use its public API and **SHOULD NOT** interfere with the underlying stream resource manually.

This class takes an optional `LoopInterface|null $loop` parameter that can be used to pass the event loop instance to use for this object. You can use a `null` value here in order to use the default loop. This value **SHOULD NOT** be given unless you're sure you want to explicitly use a given event loop instance.

This class takes an optional `int|null $readChunkSize` parameter that controls the maximum buffer size in bytes to read at once from the stream. You can use a `null` value here in order to apply its default value. This value **SHOULD NOT** be changed unless you know what you're doing. This can be a positive number which means that up to X bytes will be read at once from the underlying stream resource. Note that the actual number of bytes read may be lower if the stream resource has less than X bytes currently available. This can be `-1` which means "read everything available" from the underlying stream resource. This should read until the stream resource is not readable anymore (i.e. underlying buffer drained), note that this does not necessarily mean it reached EOF.

```
$stream = new ReadableResourceStream(STDIN, null, 8192);
```

PHP bug warning: If the PHP process has explicitly been started without a STDIN stream, then trying to read from STDIN may return data from another stream resource. This does not happen if you start this with an empty stream like `php test.php < /dev/null` instead of `php test.php <&-`. See #81 for more details.

Changelog: As of v1.2.0 the `$loop` parameter can be omitted (or skipped with a null value) to use the default loop.

WritableResourceStream

The `WritableResourceStream` is a concrete implementation of the `WritableStreamInterface` for PHP's stream resources.

This can be used to represent a write-only resource like a file stream opened in writable mode or a stream such as STDOUT or STDERR:

```
$stream = new WritableResourceStream(STDOUT);
$stream->write('hello!');
$stream->end();
```

See also `WritableStreamInterface` for more details.

The first parameter given to the constructor MUST be a valid stream resource that is opened for writing. Otherwise, it will throw an `InvalidArgumentException`:

```
// throws InvalidArgumentException
$stream = new WritableResourceStream(false);
```

See also the `DuplexResourceStream` for read-and-write stream resources otherwise.

Internally, this class tries to enable non-blocking mode on the stream resource which may not be supported for all stream resources. Most notably, this is not supported by pipes on Windows (STDOUT, STDERR etc.). If this fails, it will throw a `RuntimeException`:

```
// throws RuntimeException on Windows
$stream = new WritableResourceStream(STDOUT);
```

Once the constructor is called with a valid stream resource, this class will take care of the underlying stream resource. You SHOULD only use its public API and SHOULD NOT interfere with the underlying stream resource manually.

Any `write()` calls to this class will not be performed instantly, but will be performed asynchronously, once the `EventLoop` reports the stream resource is ready to accept data. For this, it uses an in-memory buffer string to collect all outstanding writes. This buffer has a soft-limit applied which defines how much data it is willing to accept before the caller SHOULD stop sending further data.

This class takes an optional `LoopInterface|null $loop` parameter that can be used to pass the event loop instance to use for this object. You can use a `null` value here in order to use the default loop. This value SHOULD NOT be given unless you're sure you want to explicitly use a given event loop instance.

This class takes an optional `int|null $writeBufferSoftLimit` parameter that controls this maximum buffer size in bytes. You can use a `null` value here in order to apply its default value. This value SHOULD NOT be changed unless you know what you're doing.

```
$stream = new WritableResourceStream(STDOUT, null, 8192);
```

This class takes an optional `int|null $writeChunkSize` parameter that controls this maximum buffer size in bytes to write at once to the stream. You can use a `null` value here in order to apply its default value. This value SHOULD NOT be changed unless you know what you're doing. This can be a positive number which means that up to X bytes will be written at once to the underlying stream resource. Note that the actual number of bytes written may be lower if the stream resource has less than X bytes currently available. This can be `-1` which means “write everything available” to the underlying stream resource.

```
$stream = new WritableResourceStream(STDOUT, null, null, 8192);
```

See also `write()` for more details.

Changelog: As of v1.2.0 the `$loop` parameter can be omitted (or skipped with a `null` value) to use the default loop.

DuplexResourceStream

The `DuplexResourceStream` is a concrete implementation of the `DuplexStreamInterface` for PHP's stream resources.

This can be used to represent a read-and-write resource like a file stream opened in read and write mode or a stream such as a TCP/IP connection:

```
$conn = stream_socket_client('tcp://google.com:80');
$stream = new DuplexResourceStream($conn);
$stream->write('hello!');
$stream->end();
```

See also `DuplexStreamInterface` for more details.

The first parameter given to the constructor MUST be a valid stream resource that is opened for reading *and* writing. Otherwise, it will throw an `InvalidArgumentException`:

```
// throws InvalidArgumentException
$stream = new DuplexResourceStream(false);
```

See also the `ReadableResourceStream` for read-only and the `WritableResourceStream` for write-only stream resources otherwise.

Internally, this class tries to enable non-blocking mode on the stream resource which may not be supported for all stream resources. Most notably, this is not supported by pipes on Windows (STDOUT, STDERR etc.). If this fails, it will throw a `RuntimeException`:

```
// throws RuntimeException on Windows
$stream = new DuplexResourceStream(STDOUT);
```

Once the constructor is called with a valid stream resource, this class will take care of the underlying stream resource. You **SHOULD** only use its public API and **SHOULD NOT** interfere with the underlying stream resource manually.

This class takes an optional `LoopInterface|null $loop` parameter that can be used to pass the event loop instance to use for this object. You can use a `null` value here in order to use the default loop. This value **SHOULD NOT** be given unless you're sure you want to explicitly use a given event loop instance.

This class takes an optional `int|null $readChunkSize` parameter that controls the maximum buffer size in bytes to read at once from the stream. You can use a `null` value here in order to apply its default value. This value **SHOULD NOT** be changed unless you know what you're doing. This can be a positive number which means that up to X bytes will be read at once from the underlying stream resource. Note that the actual number of bytes read may be lower if the stream resource has less than X bytes currently available. This can be `-1` which means "read everything available" from the underlying stream resource. This should read until the stream resource is not readable anymore (i.e. underlying buffer drained), note that this does not necessarily mean it reached EOF.

```
$conn = stream_socket_client('tcp://google.com:80');
$stream = new DuplexResourceStream($conn, null, 8192);
```

Any `write()` calls to this class will not be performed instantly, but will be performed asynchronously, once the `EventLoop` reports the stream resource is ready to accept data. For this, it uses an in-memory buffer string to collect all outstanding writes. This buffer has a soft-limit applied which defines how much data it is willing to accept before the caller **SHOULD** stop sending further data.

This class takes another optional `WritableStreamInterface|null $buffer` parameter that controls this write behavior of this stream. You can use a `null` value here in order to apply its default value. This value **SHOULD NOT** be changed unless you know what you're doing.

If you want to change the write buffer soft limit, you can pass an instance of `WritableResourceStream` like this:

```
$conn = stream_socket_client('tcp://google.com:80');
$buffer = new WritableResourceStream($conn, null, 8192);
$stream = new DuplexResourceStream($conn, null, null, $buffer);
```

See also `WritableResourceStream` for more details.

Changelog: As of v1.2.0 the `$loop` parameter can be omitted (or skipped with a null value) to use the default loop.

ThroughStream

The `ThroughStream` implements the `DuplexStreamInterface` and will simply pass any data you write to it through to its readable end.

```
$through = new ThroughStream();
$through->on('data', $this->expectCallableOnceWith('hello'));

$through->write('hello');
```

Similarly, the `end()` method will end the stream and emit an `end` event and then `close()` the stream. The `close()` method will close the stream and emit a `close` event. Accordingly, this can also be used in a `pipe()` context like this:

```
$through = new ThroughStream();
$source->pipe($through->pipe($dest));
```

Optionally, its constructor accepts any callable function which will then be used to *filter* any data written to it. This function receives a single data argument as passed to the writable side and must return the data as it will be passed to its readable end:

```
$through = new ThroughStream('strtoupper');
$source->pipe($through->pipe($dest));
```

Note that this class makes no assumptions about any data types. This can be used to convert data, for example for transforming any structured data into a newline-delimited JSON (NDJSON) stream like this:

```
$through = new ThroughStream(function ($data) {
    return json_encode($data) . PHP_EOL;
});
$through->on('data', $this->expectCallableOnceWith("[2, true]\n"));

$through->write(array(2, true));
```

The callback function is allowed to throw an `Exception`. In this case, the stream will emit an `error` event and then `close()` the stream.

```
$through = new ThroughStream(function ($data) {
    if (!is_string($data)) {
        throw new \UnexpectedValueException('Only strings allowed');
    }
    return $data;
});
$through->on('error', $this->expectCallableOnce());
$through->on('close', $this->expectCallableOnce());
```

```
$through->on('data', $this->expectCallableNever()));

$through->write(2);
```

CompositeStream

The `CompositeStream` implements the `DuplexStreamInterface` and can be used to create a single duplex stream from two individual streams implementing `ReadableStreamInterface` and `WritableStreamInterface` respectively.

This is useful for some APIs which may require a single `DuplexStreamInterface` or simply because it's often more convenient to work with a single stream instance like this:

```
$stdin = new ReadableResourceStream(STDIN);
$stdout = new WritableResourceStream(STDOUT);

$stdio = new CompositeStream($stdin, $stdout);

$stdio->on('data', function ($chunk) use ($stdio) {
    $stdio->write('You said: ' . $chunk);
});
```

This is a well-behaving stream which forwards all stream events from the underlying streams and forwards all streams calls to the underlying streams.

If you `write()` to the duplex stream, it will simply `write()` to the writable side and return its status.

If you `end()` the duplex stream, it will `end()` the writable side and will `pause()` the readable side.

If you `close()` the duplex stream, both input streams will be closed. If either of the two input streams emits a `close` event, the duplex stream will also close. If either of the two input streams is already closed while constructing the duplex stream, it will `close()` the other side and return a closed stream.

Usage

The following example can be used to pipe the contents of a source file into a destination file without having to ever read the whole file into memory:

```
$source = new React\Stream\ReadableResourceStream(fopen('source.txt', 'r'));
$dest = new React\Stream\WritableResourceStream(fopen('destination.txt', 'w'));

$source->pipe($dest);
```

Note that this example uses `fopen()` for illustration purposes only. This should not be used in a truly async program because the filesystem

tem is inherently blocking and each call could potentially take several seconds. See also creating streams for more sophisticated examples.

Install

The recommended way to install this library is through Composer. New to Composer?

This project follows SemVer. This will install the latest supported version:

```
composer require react/stream:^1.4
```

See also the CHANGELOG for details about version upgrades.

This project aims to run on any platform and thus does not require any PHP extensions and supports running on legacy PHP 5.3 through current PHP 8+ and HHVM. It's *highly recommended to use PHP 7+* for this project due to its vast performance improvements.

Tests

To run the test suite, you first need to clone this repo and then install all dependencies through Composer:

```
composer install
```

To run the test suite, go to the project root and run:

```
vendor/bin/phpunit
```

The test suite also contains a number of functional integration tests that rely on a stable internet connection. If you do not want to run these, they can simply be skipped like this:

```
vendor/bin/phpunit --exclude-group internet
```

License

MIT, see LICENSE file.

More

- See creating streams for more information on how streams are created in real-world applications.
- See our users wiki and the dependents on Packagist for a list of packages that use streams in real-world applications.